Store the shared parts once.
Get the full record back.

A step-by-step guide to Sourcify’s new processor representation.



The storage changes. The information survives.

JSON is a labeled description.

SCHEMATIC CONTRACT RECORD

```
{
  "address": "0x…",
  "compilation": { … },
  "sources": { … },
  "runtimeBytecode": { … },
  "match": "exact_match"
}
```

1. **Properties label the facts.**
   An address, a compiler setting, source text.
2. **Values can be large.**
   A source file may be thousands of lines.
3. **Structure carries meaning.**
   Missing, null, empty and ordered lists differ.

Full records repeat large bodies.

Record A

- Address A · verification
- Same source text
- Same compiler artifacts
- Same compiled bytecode

Record B

- Address B · verification
- Same source text
- Same compiler artifacts
- Same compiled bytecode

Record C

- Address C · verification
- Same source text
- Same compiler artifacts
- Same compiled bytecode

> The earlier format kept full JSON **even when shared entities already existed.**

Illustrative records sharing one compilation. Different records need not share these values.

Which facts belong to one record?

RECORD-SPECIFIC

Where and how it was verified

- Contract address and deployment
- Match result and verification time
- Changes between compiled and onchain code

REUSABLE CONTENT

What was compiled

- Source files and compiler settings
- ABI, metadata and compiler artifacts
- Bytecode bodies that match exactly

Different addresses can point to the same shared content.

An entity is a stored package.

ENTITY	
KEY	A unique handle for this entity
ATTRIBUTES	Labels and indexes for finding it
PAYLOAD	The data body: JSON or raw bytes

One job per package

A source-file entity stores source text.

A code entity stores bytecode bytes.

A compilation entity connects reusable compiler data.

A verification entity connects the complete record.

A hash is a compact content fingerprint.



Exact source text

Same UTF-8 bytes

SHA-256

→

ACTUAL SAMPLE SOURCE HASH · ABBREVIATED `0xa32e2bb41a928c…`

Used to find and validate the shared body.

Source text

SHA-256 of UTF-8 content

Bytecode

Keccak-256 of raw bytes

Compilation

SHA-256 of its canonical v3 envelope

Different hash roles are not interchangeable. A matching code hash does not mean an identical verification.

A reference says where to find it and where to put it back.

BEFORE

A large inline value

Repeated source content

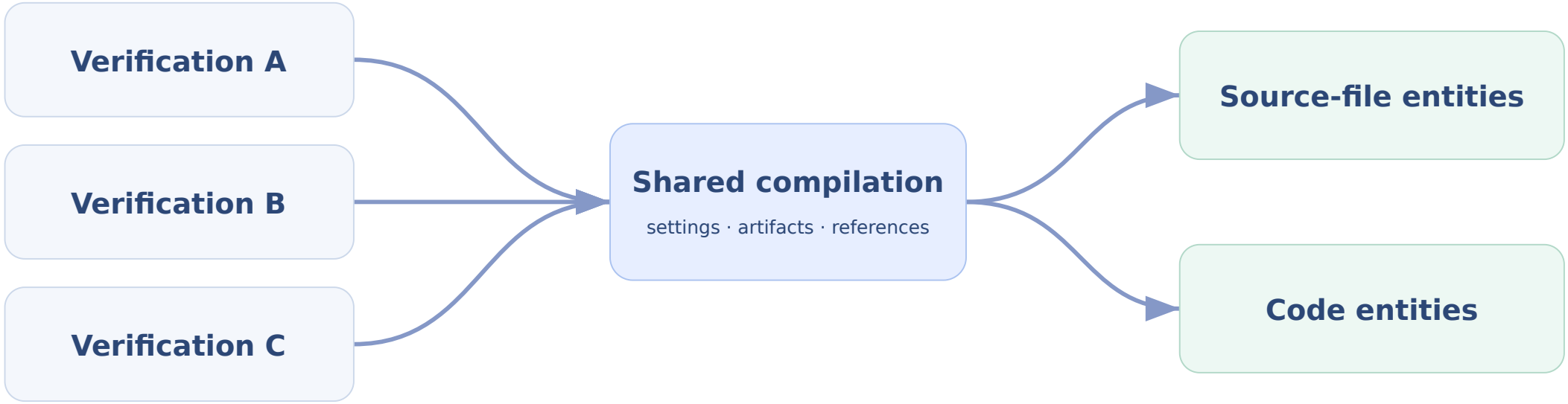
→

STORED

Path	Destination in the original JSON
Kind	Which kind of entity to read
Hash	Which exact shared content

The value moves into shared storage. **It is not discarded.**

Many records can share one compilation.



Share exact bodies; keep each record’s own facts.

Illustrative relationships. Onchain bytecode can also be referenced directly by verification records.

Stored JSON is stripped. Hydrated JSON is complete.

WHAT WE STORE

Versioned envelope

```
{
  "schema": "…v3",
  "data": { … },
  "references": [ … ]
}
```

Local values + instructions to restore shared values

WHAT THE READER RETURNS

Original contract JSON

```
{
  "address": "0x…",
  "sources": { … },
  "compilation": { … },
  …
}
```

All original fields, in their original locations

Schematic excerpts. Full hydrated JSON is a reader result, not another stored copy.

Hydration follows the instructions.

1

Read

Load the stored record and its references.

→

2

Decode

Unpack compression and any large-payload parts.

→

3

Validate

Find dependencies and check their hashes.

→

4

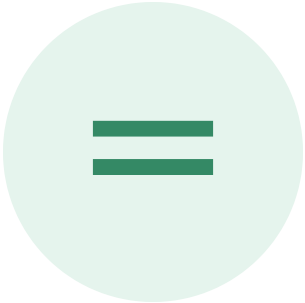
Restore

Put every value back at its original path.

```
arkiv-entity-processor hydrate BUNDLE_DIR
```

Missing or corrupt dependencies cause an error.

Equal JSON values.
Not identical text formatting.



Original input ↔ hydrated output

- Exact strings
- Null vs missing
- Array order
- Hex casing
- Unknown fields
- Exact numeric values

Preserved

The content and structure supplied to the processor.

May differ

Whitespace between JSON tokens and object-key order.

Whitespace inside a string remains part of the value and must be preserved.

SafeProxy: the full round trip passes.

CAPTURED API INPUT · ETHEREUM

1 input record → 6 actual entities

0x000000000034065b3a94C2118CFe5B4C0067B615

✓ **Hydrated JSON equals source**

1	verification
1	compilation
1	source file
3	code bodies

17,630 B
All decoded entity payloads

5,457 B
All wire payloads, after compression

12,647 B
Complete seed JSONL, with framing

Locally generated bundle, not a deployed Arkiv response. Capture and validation evidence ↗

A controlled test removes repeated bodies.

SYNTHETIC: 20 copies of the SafeProxy record; only address and matchId changed.

Gzip wire payload bytes

Earlier full-record format

69,327 B

Shared v3 format

23,662 B

25 entities in both

1 shared compilation in both

20 / 20

Hydrated records equal their inputs

Complete seed JSONL

126,242 → 65,326 B

≈48% smaller complete JSONL

The gain comes from fewer repeated bytes, not fewer entities.

This is not 20 independently observed contracts and not a full-dataset savings estimate. Method and exact figures ↗

Inspect the change.
Then update readers and measure at scale.

Three clicks in the live example

1. **Stored / stripped**
   Blue “Referenced” rows show shared values.
2. **Follow a reference**
   Inspect the actual source, code or compilation.
3. **Full hydrated**
   Turn on “Differences only”: no differences.

Open stored view ↗ Open hydrated view ↗

Evidence: real example · controlled benchmark · format and hashes · implementation sources

What remains

Readers: support v3 before consuming regenerated bundles.

Scale: measure representative data; no corpus savings claim yet.

Input limits: offline exports lack live proxy resolution and can differ in metadata text.